CHAPTER 1

Deep Learning Foundations

The fundamental challenge of artificial intelligence resides in discovering computational mechanisms capable of mapping high-dimensional, unstructured sensory and symbolic observations into structured semantic representations. Classical machine learning systems approached this objective by decoupling feature engineering from statistical inference: domain experts manually designed deterministic transformation pipelines to extract handcrafted numerical descriptors, which were subsequently supplied to shallow linear classifiers or regression models. While computationally tractable in controlled environments, this paradigm proved fragile when confronted with the continuous variability, high dimensionality, and compositional complexity of natural language and perceptual signals.

Deep learning fundamentally alters this architectural philosophy by adopting a connectionist, data-driven formulation. Rather than relying on human engineering to prescribe fixed representational primitives, deep neural networks learn hierarchical feature representations directly from raw data through the end-to-end composition of parameterized, differentiable mathematical functions. Each successive layer computes an abstract geometric transformation of its input, constructing intermediate latent representations that disentangle the underlying factors of variation.

This chapter establishes the theoretical, geometric, and algorithmic foundations of deep feed-forward architectures. We trace the lineage of neural computation from early biological abstractions and the single-layer perceptron, formalize the geometric barriers imposed by linear decision boundaries, derive the space-folding mechanics of non-linear activation functions, establish the statistical grounding of task-specific loss functions under maximum likelihood estimation, and formulate reverse-mode automatic differentiation on computational graphs as the universal optimization engine for deep learning.

1.1 Historical Context and Theoretical Origins

Understanding the structural necessity of deep neural networks requires examining why classical algorithmic paradigms and shallow statistical models proved insufficient for complex perceptual and linguistic tasks. The progression from symbolic reasoning to modern representation learning illustrates how computational and geometric constraints shaped modern neural architectures.

1.1.1 From symbolic artificial intelligence to connectionism

For several decades following the Dartmouth workshop of 1956, artificial intelligence research was dominated by symbolic systems, colloquially termed Good Old-Fashioned Artificial Intelligence (GOFAI) [**?**]. This paradigm operated on the physical symbol system hypothesis, positing that intelligent behavior could be instantiated through the formal manipulation of discrete, explicit symbols according to hand-engineered logical rules. Knowledge was encoded using production systems, semantic networks, and first-order predicate calculus, where inference engines evaluated deterministic truth values over closed ontologies.

While symbolic systems demonstrated notable competence in closed, deterministic micro-worlds such as chess or theorem proving, they encountered severe operational bottlenecks when deployed in real-world environments characterized by noise, perceptual ambiguity, and continuous variations. Hand-crafted rule bases were inherently brittle: any scenario outside the codified axioms triggered catastrophic failure, and resolving contradictory evidence in large rule graphs became computationally intractable. More critically, symbolic systems lacked an intrinsic metric of semantic similarity; the discrete symbols `cat` and `feline` shared no mathematical relationship in coordinate space, precluding continuous generalization.

In response to these limitations, the connectionist paradigm emerged, proposing that cognitive processing is realized through distributed computation over vast networks of simple, interconnected processing units [**?** **?** **?**]. Rather than storing knowledge in localized, explicit symbolic rules, connectionist networks encode knowledge in the continuous, distributed pattern of synaptic connection strengths (weights) across the entire system. Representation and computation are inherently continuous and differentiable, enabling models to interpolate smoothly across input spaces, tolerate corrupt or incomplete inputs through graceful degradation, and acquire representations directly from empirical data through statistical learning.

1.1.2 The curse of dimensionality and high-dimensional geometry

Early statistical approaches to non-linear pattern recognition attempted to circumvent the limitations of linear models by projecting input vectors $\mathbf{x} \in \mathbb{R}^D$ into higher-dimensional feature spaces via fixed basis functions $\phi(\mathbf{x}) : \mathbb{R}^D \to \mathbb{R}^M$. A standard linear model was then evaluated on the transformed representations:

$$y(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \phi(\mathbf{x}) = \sum_{j=1}^{M} w_j \phi_j(\mathbf{x}) + w_0, \tag{1.1}$$

where $\mathbf{w} \in \mathbb{R}^M$ denotes the parameter vector and w_0 represents the scalar bias [**?**]. Although this formulation preserves linear parameter optimization, it suffers directly from the curse of dimensionality when scaling to high-dimensional input spaces.

Consider constructing a complete polynomial basis of order P over an input dimension D. The total number of independent polynomial basis functions required scales asymptotically as:

$$M = \binom{D+P}{P} = \mathcal{O}\left(D^P\right). \tag{1.2}$$

For an input space of moderate dimensionality, such as $D = 100$, a third-order polynomial expansion ($P = 3$) demands $M \approx 1.7 \times 10^5$ parameters, while a fifth-order polynomial ($P = 5$) requires $M \approx 7.5 \times 10^7$ parameters. When processing natural images or tokenized textual contexts where $D \geq 10^3$, the parameter count exceeds any feasible computational or sample budget.

The curse of dimensionality manifests with equal severity in non-parametric and partitioning methods. Suppose the input domain $[0, 1]^D$ is partitioned into regular grid cells by dividing each dimension into K intervals. The total number of hypercubical cells is:

$$N_{\text{cells}} = K^D. \tag{1.3}$$

If $D = 50$ and each axis is divided into only $K = 10$ bins, the volume is partitioned into 10^{50} distinct cells. Because any realistic dataset contains $N \ll 10^{50}$ observations, the overwhelming majority of cells will remain completely unpopulated, causing localized neighborhood estimators to fail through severe sample sparsity.

Furthermore, the geometry of high-dimensional Euclidean space exhibits counter-intuitive properties that undermine distance-based algorithms [**?**]. Consider a hypersphere of radius $r = 1$ in D dimensions. The volume of the hypersphere is proportional to r^D. The fraction of this volume lying within a thin spherical shell of thickness $\epsilon \in (0, 1)$ near the outer surface is given by:

$$\frac{V_D(1) - V_D(1 - \epsilon)}{V_D(1)} = 1 - (1 - \epsilon)^D. \tag{1.4}$$

As the dimensionality $D \to \infty$, the term $(1 - \epsilon)^D \to 0$, which causes the volume fraction to converge asymptotically to 1 for any non-zero ϵ:

$$\lim_{D \to \infty} \left[1 - (1 - \epsilon)^D\right] = 1. \tag{1.5}$$

Consequently, in high-dimensional spaces, nearly all the volume of a hypersphere is concentrated in an infinitesimally thin boundary shell. A complementary phenomenon occurs with multivariate Gaussian distributions $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$: transforming to spherical polar coordinates and integrating over directional variables reveals that the radial probability density $p(r) \propto r^{D-1}e^{-r^2/(2\sigma^2)}$ concentrates entirely within a narrow spherical shell of radius $r \approx \sigma\sqrt{D}$. Data points in high dimensions do not cluster around the mean; they reside along an equatorial hypersphere, causing pairwise Euclidean distances between random observations to concentrate tightly around a single constant value.

1.1.3 The manifold hypothesis and the expressive power of depth

The realization that high-dimensional data points become mutually equidistant under isotropic assumptions led to the formulation of the manifold hypothesis [**? ?**]. The hypothesis states that natural high-dimensional datasets (such as natural language sentences, audio waveforms, or natural images) do not populate the ambient Euclidean space $\mathbb{R}^D$ uniformly; rather, they concentrate in the immediate vicinity of a low-dimensional, non-linear embedded sub-manifold $\mathcal{M} \subset \mathbb{R}^D$ having intrinsic dimension $d \ll D$.

Fixed basis function models cannot exploit this intrinsic low-dimensional structure because their basis functions $\phi_j(\mathbf{x})$ are chosen prior to observing the data distribution. To model arbitrary functions over the ambient space, fixed basis methods must tile the entire D-dimensional volume uniformly, falling victim to the exponential scaling of K^D. Conversely, artificial neural networks resolve this impasse by replacing fixed basis functions with parameterized, data-dependent basis functions that are optimized jointly to conform to the geometry of the data manifold.

The capacity of neural networks to parameterize complex manifolds depends critically on network depth. Consider a feed-forward network utilizing Rectified Linear Unit (ReLU)

activation functions. Because the ReLU function is piecewise linear, any network constructed from ReLU activations computes a continuous, piecewise linear function that partitions the input domain $\mathbb{R}^{d_{\text{in}}}$ into a collection of convex polyhedral regions, within each of which the model evaluates a distinct affine mapping [**?** **?**].

For a shallow network comprising a single hidden layer with K hidden units and a scalar input ($d_{\text{in}} = 1$), the model can construct at most $K + 1$ linear regions, parameterized by $3K + 1$ weights and biases:

$$y = \phi_0 + \sum_{j=1}^{K} \phi_j \max\left(0, \theta_{j0} + \theta_{j1} x\right). \tag{1.6}$$

To increase the number of linear regions in a shallow architecture, one must expand the hidden width linearly: achieving $O(K)$ linear regions requires $O(K)$ parameters.

In sharp contrast, stacking multiple non-linear layers allows deeper layers to compute piecewise linear compositions of the piecewise linear functions evaluated by preceding layers. Each successive layer folds the input space, replicating the linear partitions constructed by earlier stages. Formalized by Montúfar et al. [**?**], a deep neural network with L hidden layers, each possessing K hidden units, can partition the input space $\mathbb{R}^{d_{\text{in}}}$ into a number of distinct linear regions that scales exponentially with depth:

$$N_{\text{regions}} \geq \left(\left\lfloor \frac{K}{d_{\text{in}}} \right\rfloor\right)^{(L-1)d_{\text{in}}} K^{d_{\text{in}}} = O\left(K^{L \cdot d_{\text{in}}}\right), \tag{1.7}$$

while the total parameter count grows only linearly with depth and quadratically with width:

$$N_{\text{parameters}} = O\left(L \cdot K^2\right). \tag{1.8}$$

This exponential disparity in expressive efficiency provides the fundamental mathematical justification for deep architectures: deep composition generates exponentially complex, multi-scale decision boundaries using a polynomial parameter budget, enabling neural networks to model intricate manifold topologies that remain inaccessible to shallow models.

1.2 The Perceptron and Linear Separability

The conceptual building block of modern neural architectures is the artificial perceptron. To understand why modern deep architectures are structured as hierarchical non-linear networks, one must first analyze the computational properties and geometric limitations of this foundational linear operator.

1.2.1 The artificial perceptron

The mathematical formulation of the artificial neuron originated in the simplified physiological model of biological neurons proposed by Warren McCulloch and Walter Pitts in 1943 [**?**], which Frank Rosenblatt subsequently formalized and operationalized as the perceptron learning model in 1958 [**?**]. The biological abstraction captures three primary mechanisms: dendrites receive incoming electrochemical impulses, the soma integrates these inputs, and the axon fires an action potential if the accumulated potential surpasses an excitation threshold.

Formally, consider an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^\top \in \mathbb{R}^n$. Each input component x_i is scaled by an associated scalar weight $w_i \in \mathbb{R}$, which quantifies the synaptic efficacy of that specific connection. The neuron aggregates these weighted contributions and adds an intrinsic scalar parameter termed the bias, $b \in \mathbb{R}$, yielding a scalar pre-activation value $z \in \mathbb{R}$:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b, \tag{1.9}$$

where $\mathbf{w} = [w_1, w_2, \ldots, w_n]^\top \in \mathbb{R}^n$ represents the weight vector. The pre-activation value is subsequently processed by an activation function $f : \mathbb{R} \to \mathcal{Y}$ to produce the final scalar output y:

$$y = f(z) = f\left(\mathbf{w}^\top \mathbf{x} + b\right). \tag{1.10}$$

In Rosenblatt's classical formulation, the activation function is instantiated as the discontinuous Heaviside step function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0, \\ 0 & \text{if } z < 0, \end{cases} \tag{1.11}$$

producing a binary decision variable indicating class membership.

To streamline the algebraic notation, the formulation can be expressed homogeneously by augmenting the input vector with a dummy feature $x_0 = 1$ and defining an augmented weight vector $\tilde{\mathbf{w}} = [w_0, w_1, \ldots, w_n]^\top \in \mathbb{R}^{n+1}$ where $w_0 = b$. Under this homogeneous convention, Equation (1.9) simplifies to a single inner product:

$$z = \sum_{i=0}^{n} w_i x_i = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}, \tag{1.12}$$

where $\tilde{\mathbf{x}} = [1, x_1, \ldots, x_n]^\top \in \mathbb{R}^{n+1}$.

1.2.2 The geometry of decision hyperplanes

The operational behavior of the artificial perceptron is governed by the geometric properties of its decision boundary in the Euclidean input space $\mathbb{R}^n$. The decision boundary corresponds to the locus of points where the pre-activation vanishes identically:

$$\mathcal{H} = \left\{ \mathbf{x} \in \mathbb{R}^n \;\middle|\; \mathbf{w}^\top \mathbf{x} + b = 0 \right\}. \tag{1.13}$$

In an n-dimensional space, Equation (1.13) defines an $(n-1)$-dimensional affine subspace termed a hyperplane. In a two-dimensional input space ($n = 2$), the hyperplane reduces to a standard straight line:

$$w_1 x_1 + w_2 x_2 + w_0 = 0 \iff x_2 = -\frac{w_1}{w_2} x_1 - \frac{w_0}{w_2}, \tag{1.14}$$

assuming $w_2 \neq 0$.

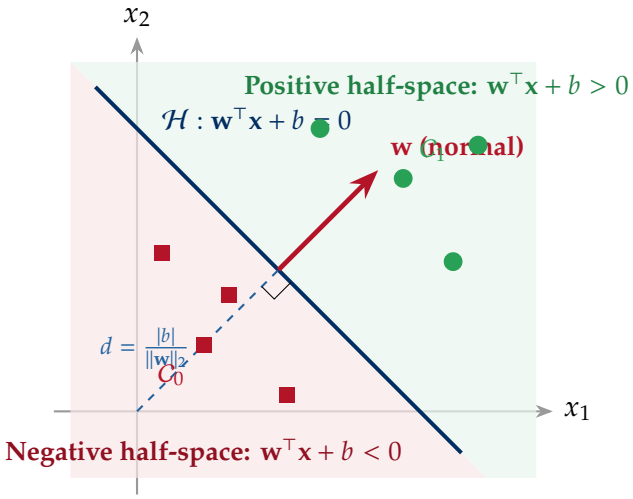


Figure 1.1: Geometric partitioning of a two-dimensional input space by an artificial perceptron decision hyperplane $\mathcal{H}$. The parameter vector $\mathbf{w}$ is strictly orthogonal to the decision boundary, while the scalar bias b determines the orthogonal translation of the hyperplane relative to the origin.

The geometric orientation and placement of the hyperplane are governed by two mathematical properties:

(i) **Orthogonality of the weight vector**: The parameter vector $\mathbf{w}$ is strictly orthogonal to every vector lying within the hyperplane $\mathcal{H}$. To verify this, consider two arbitrary points $\mathbf{x}_A, \mathbf{x}_B \in \mathcal{H}$. By definition of the hyperplane:

$$\mathbf{w}^\top \mathbf{x}_A + b = 0 \quad \text{and} \quad \mathbf{w}^\top \mathbf{x}_B + b = 0. \tag{1.15}$$

Subtracting the two equalities yields:

$$\mathbf{w}^\top (\mathbf{x}_A - \mathbf{x}_B) = 0. \tag{1.16}$$

Because the difference vector $\mathbf{v} = \mathbf{x}_A - \mathbf{x}_B$ is parallel to the hyperplane and its inner product with $\mathbf{w}$ vanishes, $\mathbf{w}$ is normal to the decision boundary.

(ii) **Orthogonal distance to the boundary**: The signed orthogonal distance $d(\mathbf{x}, \mathcal{H})$ from an arbitrary point $\mathbf{x} \in \mathbb{R}^n$ to the hyperplane $\mathcal{H}$ is obtained by projecting the displacement vector $(\mathbf{x} - \mathbf{x}_p)$, where $\mathbf{x}_p \in \mathcal{H}$, onto the unit normal vector $\frac{\mathbf{w}}{\|\mathbf{w}\|_2}$:

$$d(\mathbf{x}, \mathcal{H}) = \frac{\mathbf{w}^\top (\mathbf{x} - \mathbf{x}_p)}{\|\mathbf{w}\|_2} = \frac{\mathbf{w}^\top \mathbf{x} - \mathbf{w}^\top \mathbf{x}_p}{\|\mathbf{w}\|_2} = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|_2}. \tag{1.17}$$

Evaluating this expression at the origin ($\mathbf{x} = \mathbf{0}$) yields the perpendicular distance from the coordinate origin to the hyperplane:

$$d_{\text{origin}} = \frac{|b|}{\|\mathbf{w}\|_2}. \tag{1.18}$$

The perceptron partitions the input space into two half-spaces: a positive half-space where $\mathbf{w}^\top \mathbf{x} + b > 0$ (yielding $y = 1$) and a negative half-space where $\mathbf{w}^\top \mathbf{x} + b < 0$ (yielding $y = 0$).

1.2.3 The linear stacking collapse proof

Given the limitations of a single linear decision boundary, an intuitive proposal involves cascading multiple perceptrons into a multi-layer hierarchical network, where intermediate units supply inputs to subsequent stages. However, if the activation functions between successive layers remain purely linear (that is, $f(z) = z$), the multi-layer architecture collapses algebraically into an ordinary single-layer linear model.

Proof of linear stacking collapse. Consider a multi-layer feed-forward network comprising L fully connected linear layers. Let $\mathbf{x} \in \mathbb{R}^{d_0}$ denote the input vector. The first hidden layer computes an affine transformation parameterized by weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_0 \times d_1}$ and bias vector $\mathbf{b}_1 \in \mathbb{R}^{d_1}$:

$$\mathbf{h}_1 = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1. \tag{1.19}$$

The second hidden layer processes $\mathbf{h}_1$ linearly via $\mathbf{W}_2 \in \mathbb{R}^{d_1 \times d_2}$ and $\mathbf{b}_2 \in \mathbb{R}^{d_2}$:

$$\mathbf{h}_2 = \mathbf{W}_2^\top \mathbf{h}_1 + \mathbf{b}_2 = \mathbf{W}_2^\top \left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2. \tag{1.20}$$

Expanding this expression by distributing the matrix multiplication yields:

$$\mathbf{h}_2 = (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + \left(\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2\right). \tag{1.21}$$

By defining an effective composite weight matrix $\mathbf{W}' \in \mathbb{R}^{d_0 \times d_2}$ and an effective composite bias vector $\mathbf{b}' \in \mathbb{R}^{d_2}$:

$$\mathbf{W}' = \mathbf{W}_1 \mathbf{W}_2 \quad \text{and} \quad \mathbf{b}' = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2, \tag{1.22}$$

Equation (1.21) simplifies to:

$$\mathbf{h}_2 = \mathbf{W}'^\top \mathbf{x} + \mathbf{b}'. \tag{1.23}$$

By mathematical induction, compounding L successive linear layers yields:

$$\mathbf{y} = \mathbf{W}_{\text{eff}}^\top \mathbf{x} + \mathbf{b}_{\text{eff}}, \quad \text{where} \quad \mathbf{W}_{\text{eff}} = \prod_{l=1}^{L} \mathbf{W}_l. \tag{1.24}$$

Because the product of L linear operators is itself a linear operator, stacking arbitrary numbers of purely linear layers provides zero additional expressive capacity over a single affine layer. Consequently, depth without non-linearity is functionally vacuous.

1.2.4 The XOR linear inseparability barrier

The catastrophic consequence of the linear restriction was formally exposed by Marvin Minsky and Seymour Papert in their seminal 1969 monograph [**?**], which demonstrated that a single-layer perceptron is incapable of computing the basic exclusive-OR (XOR) logic function.

The XOR truth table maps two binary inputs $x_1, x_2 \in \{0, 1\}$ to a scalar output $y \in \{0, 1\}$ according to:

$$\text{XOR}(0,0) = 0, \quad \text{XOR}(0,1) = 1, \quad \text{XOR}(1,0) = 1, \quad \text{XOR}(1,1) = 0. \tag{1.25}$$

For a perceptron with threshold activation $f(z) = \mathbf{1}_{z>0}$ to compute this function, there must exist real parameters $w_1, w_2, b \in \mathbb{R}$ satisfying four simultaneous linear inequalities:

$$(x_1 = 0, x_2 = 0) \implies w_1(0) + w_2(0) + b \leq 0 \iff b \leq 0, \tag{1.26}$$
$$(x_1 = 0, x_2 = 1) \implies w_1(0) + w_2(1) + b > 0 \iff w_2 + b > 0, \tag{1.27}$$
$$(x_1 = 1, x_2 = 0) \implies w_1(1) + w_2(0) + b > 0 \iff w_1 + b > 0, \tag{1.28}$$
$$(x_1 = 1, x_2 = 1) \implies w_1(1) + w_2(1) + b \leq 0 \iff w_1 + w_2 + b \leq 0. \tag{1.29}$$

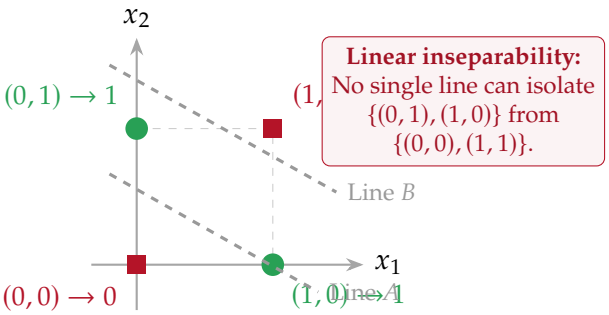


Figure 1.2: The XOR linear inseparability barrier. The two input classes alternate diagonally across the vertices of the unit square, preventing any single linear hyperplane from isolating Class 1 from Class 0.

Proof of algebraic contradiction. Summing the strict inequalities (1.27) and (1.28) yields:

$$(w_2 + b) + (w_1 + b) > 0 \iff w_1 + w_2 + 2b > 0. \tag{1.30}$$

Conversely, summing inequalities (1.26) and (1.29) produces:

$$(w_1 + w_2 + b) + b \leq 0 + 0 \iff w_1 + w_2 + 2b \leq 0. \tag{1.31}$$

Inequality (1.30) and Inequality (1.31) are mutually contradictory: the scalar quantity $w_1 + w_2 + 2b$ cannot simultaneously be strictly positive and non-positive ($0 < w_1 + w_2 + 2b \leq 0$). Hence, no parameters $\{w_1, w_2, b\}$ exist that solve the system. Because the input points alternate diagonally across the vertices of the unit square, the dataset is linearly inseparable, establishing an insurmountable barrier for purely linear units.

1.3 Activation Functions and Non-Linearity

To overcome the linear stacking collapse and solve non-linearly separable problems such as XOR, neural networks must interleave affine transformations with point-wise non-linear mapping functions. These mappings, termed activation functions, provide the mathematical mechanism that enables multi-layer networks to approximate arbitrary functional relationships.

1.3.1 Coordinate space folding and the resolution of XOR

Non-linear activation functions resolve the linear inseparability barrier by non-linearly warping and folding the input coordinate space. When input points are projected through an intermediate non-linear layer, the underlying geometric distances are altered such that points belonging to different classes, initially entangled in the original space, become linearly separable in the transformed hidden space [**?**].

To illustrate this mechanism concretely, we examine the analytical solution to the XOR problem using a two-layer Multi-Layer Perceptron (MLP) equipped with Rectified Linear Unit (ReLU) activations [**?**]:

$$\mathbf{h} = g\left(\mathbf{W}^\top \mathbf{x} + \mathbf{c}\right), \quad y = \mathbf{w}^\top \mathbf{h} + b, \tag{1.32}$$

where $g(z) = \max(0, z)$ applies element-wise. Let the network parameters be configured as:

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b = 0. \tag{1.33}$$

The hidden layer pre-activation vector is given by:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{c} = \begin{bmatrix} x_1 + x_2 \\ x_1 + x_2 - 1 \end{bmatrix}. \tag{1.34}$$

Applying the non-linear ReLU activation $g(\mathbf{z})$ evaluates the hidden representations $\mathbf{h} = [h_1, h_2]^\top$ for all four input patterns:

$$\mathbf{x} = [0,0]^\top \implies \mathbf{z} = [0,-1]^\top \implies \mathbf{h} = [\max(0,0), \max(0,-1)]^\top = [0,0]^\top, \tag{1.35}$$
$$\mathbf{x} = [0,1]^\top \implies \mathbf{z} = [1,0]^\top \implies \mathbf{h} = [\max(0,1), \max(0,0)]^\top = [1,0]^\top, \tag{1.36}$$
$$\mathbf{x} = [1,0]^\top \implies \mathbf{z} = [1,0]^\top \implies \mathbf{h} = [\max(0,1), \max(0,0)]^\top = [1,0]^\top, \tag{1.37}$$
$$\mathbf{x} = [1,1]^\top \implies \mathbf{z} = [2,1]^\top \implies \mathbf{h} = [\max(0,2), \max(0,1)]^\top = [2,1]^\top. \tag{1.38}$$

The geometric effect of the non-linearity is readily visible: the inputs $(0,1)$ and $(1,0)$, which occupied disjoint coordinates in the original input space, are mapped to the identical latent coordinate $\mathbf{h} = [1,0]^\top$.

The output layer computes a linear combination of the hidden coordinates:

$$y = \mathbf{w}^\top \mathbf{h} + b = 1 \cdot h_1 - 2 \cdot h_2 + 0. \tag{1.39}$$

Evaluating the output on each transformed latent point yields:

$$\mathbf{x} = [0,0]^\top \implies y = 1(0) - 2(0) = 0, \tag{1.40}$$
$$\mathbf{x} = [0,1]^\top \implies y = 1(1) - 2(0) = 1, \tag{1.41}$$
$$\mathbf{x} = [1,0]^\top \implies y = 1(1) - 2(0) = 1, \tag{1.42}$$
$$\mathbf{x} = [1,1]^\top \implies y = 1(2) - 2(1) = 0. \tag{1.43}$$

The model reproduces the XOR truth table with exact precision. By collapsing $(0,1)$ and $(1,0)$ onto a shared latent location while projecting $(1,1)$ away along the second dimension ($h_2 = 1$), the non-linear activation folded the coordinate space, transforming a linearly inseparable problem into a linearly separable representation.

1.3.2 The universal approximation theorem

The theoretical expressive power conferred by non-linear hidden units is formalized by the Universal Approximation Theorem (UAT), proved independently by George Cybenko (1989) for sigmoidal activations [**?**] and Kurt Hornik, Maxwell Stinchcombe, and Halbert White (1989) for general non-linearities [**?**].

Theorem 1.1: *Universal approximation theorem*

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous, bounded, non-constant activation function. Let $I_n = [0,1]^n$ denote the compact n-dimensional unit hypercube, and let $C(I_n)$ denote the space of continuous real-valued functions on I_n. Then, for any target function $f \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exists an integer $M \in \mathbb{N}$, a set of real coefficients $v_j \in \mathbb{R}$, weight vectors $\mathbf{w}_j \in \mathbb{R}^n$, and biases $b_j \in \mathbb{R}$ such that the finite sum:

$$F(\mathbf{x}) = \sum_{j=1}^{M} v_j \sigma\left(\mathbf{w}_j^\top \mathbf{x} + b_j\right) \tag{1.44}$$

satisfies:

$$\sup_{\mathbf{x} \in I_n} \left|F(\mathbf{x}) - f(\mathbf{x})\right| < \epsilon. \tag{1.45}$$

In functional analysis terms, the family of single-hidden-layer feed-forward networks is dense in $C(I_n)$ with respect to the uniform supremum metric.

Constructive intuition via localized bump functions. The mathematical mechanism underlying Cybenko's proof can be understood through the constructive superposition of localized rectangular "bump" functions. Consider a one-dimensional input $x \in \mathbb{R}$ and a standard logistic sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$. By taking the difference between two sigmoids displaced by a distance $\Delta = b - a$ and scaled by an arbitrary steepness parameter $w \gg 0$, one constructs a localized pulse or bump function:

$$B(x; a, b, w) = \sigma(w(x - a)) - \sigma(w(x - b)). \tag{1.46}$$

As the scaling factor $w \to \infty$, the sigmoid transitions approximate ideal Heaviside steps, causing $B(x)$ to converge to a rectangular gate function that equals 1 on the interval (a, b) and vanishes outside:

$$\lim_{w \to \infty} B(x; a, b, w) = \begin{cases} 1 & \text{if } x \in (a, b), \\ 0 & \text{otherwise.} \end{cases} \tag{1.47}$$

In an n-dimensional domain, intersecting $2n$ such sigmoidal half-spaces constructs an n-dimensional hypercubical localized bump function that isolates a localized neighborhood of volume ϵ^n. By scaling the amplitude of each bump to match the local value of the target function $f(\mathbf{x})$ and tiling the compact domain I_n with a finite collection of such bumps, the network constructs a piecewise-constant Riemann approximation of $f(\mathbf{x})$. Because continuous functions on compact domains are uniformly continuous, taking sufficiently fine partitions guarantees that the approximation error remains strictly bounded below ϵ.

While the Universal Approximation Theorem provides a foundational mathematical guarantee, two critical caveats govern its practical interpretation:

(i) **Existence versus learnability**: The theorem is purely an existence proof. It guarantees that a configuration of parameters exists that approximates f, but provides no assurance that gradient-based optimization algorithms (such as gradient descent) will successfully discover those parameters from finite, noisy training samples.

(ii) **Width versus depth efficiency**: The theorem guarantees approximation using a single hidden layer, but the required hidden width M may scale exponentially with the input dimension n. As established in Section 1.1.3, deep hierarchical composition constructs multi-scale piecewise linear partitions using polynomial parameter budgets, rendering deep architectures exponentially more parameter-efficient than shallow networks of equivalent capacity.

1.3.3 Taxonomy of activation functions

The choice of non-linear activation function directly influences gradient flow, convergence velocity, and representational capacity across deep networks. Table 1.1 summarizes the mathematical formulations, analytical derivatives, output ranges, and primary failure modes of the canonical activation functions.

Table 1.1: Comparative taxonomy of modern neural activation functions.

Function	Formulation $\sigma(z)$	Derivative $\frac{d\sigma}{dz}$	Range	Zero-Centered	Primary Failure Mode
Sigmoid	$\frac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	$(0,1)$	No	Severe gradient saturation; zig-zag updates
Tanh	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$1-\tanh^2(z)$	$(-1,1)$	Yes	Gradient saturation at extreme inputs
ReLU	$\max(0,z)$	$\mathbf{1}_{z>0}$	$[0,\infty)$	No	"Dying ReLU" (inactive negative half-plane)
Leaky ReLU	$\max(\alpha z, z)$	$\mathbf{1}_{z>0}+\alpha\mathbf{1}_{z\leq 0}$	$(-\infty,\infty)$	Partial	Hyperparameter sensitivity of slope α
GELU	$z\Phi(z)$	$\Phi(z)+z\phi(z)$	$(-0.17,\infty)$	Partial	Higher transcendental computational cost
SwiGLU	$\text{Swish}(\mathbf{xW})\otimes(\mathbf{xV})$	Gated Jacobian	$(-\infty,\infty)$	Partial	Increases parameter overhead by 1.5×
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	$S_i(\delta_{ij}-S_j)$	$(0,1)$	No	Sensitive to logit scale; $O(\lvert\mathcal{V}\rvert)$ cost

Logistic sigmoid

The logistic sigmoid function squashes the entire real line into a bounded interval:

$$\sigma(z)=\frac{1}{1+e^{-z}}=\frac{e^z}{e^z+1}. \tag{1.48}$$

Its first derivative can be written directly in terms of its function values:

$$\frac{d\sigma}{dz}=\frac{e^{-z}}{(1+e^{-z})^2}=\left(\frac{1}{1+e^{-z}}\right)\left(\frac{e^{-z}}{1+e^{-z}}\right)=\sigma(z)\left(1-\sigma(z)\right). \tag{1.49}$$

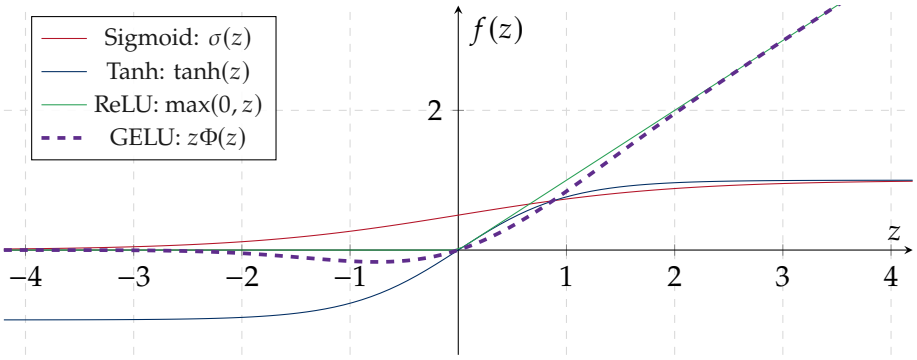


Figure 1.3: Comparison of primary activation functions across the operational domain $z \in [-4, 4]$.

Because the output is bounded in $(0, 1)$, the sigmoid function maps real logits to valid Bernoulli probabilities, making it the canonical choice for binary classification output layers.

However, when employed within deep hidden layers, the sigmoid function exhibits two severe pathologies:

(i) **Gradient saturation**: The maximum value of the derivative occurs at the origin, where $\sigma'(0) = 0.5(1 - 0.5) = 0.25$. When the magnitude of the pre-activation becomes moderately large ($|z| > 4$), the derivative rapidly decays to zero ($\sigma'(z) \approx 0$). During backpropagation, error gradients are scaled by this factor; compounding multiple values bounded above by 0.25 causes the backpropagated gradient to vanish exponentially with depth.

(ii) **Non-zero-centered outputs**: Because $\sigma(z) \in (0, 1)$, the activations entering subsequent layers are strictly positive ($a_i > 0$). In a linear transformation $z = \sum_i w_i a_i + b$, the gradient of the loss with respect to weight w_i is $\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} a_i$. Because all $a_i > 0$, the partial derivatives for all incoming weights to a unit share the exact sign of $\frac{\partial \mathcal{L}}{\partial z}$. Consequently, parameter updates can only move simultaneously in all-positive or all-negative directions, inducing inefficient zig-zag optimization trajectories.

Hyperbolic tangent

The hyperbolic tangent (tanh) function resolves the non-zero-centered limitation of the sigmoid by mapping inputs into the symmetric, zero-centered interval $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1. \tag{1.50}$$

Its first derivative evaluates to:

$$\frac{d \tanh(z)}{dz} = 1 - \tanh^2(z). \tag{1.51}$$

At the origin, the derivative evaluates to $\tanh'(0) = 1.0$, which is four times larger than that of the sigmoid function, facilitating stronger initial gradient propagation. Furthermore, because outputs have an expected value near zero under symmetric weight initializations, the covariance shifts that produce zig-zag gradient dynamics are mitigated. Nonetheless, the function remains susceptible to saturation plateaus: when $|z| \geq 3$, $|\tanh(z)| \approx 1$ and $\tanh'(z) \to 0$, causing gradients to vanish in deep architectures.

Rectified linear unit

The Rectified Linear Unit (ReLU), popularized in deep learning by Glorot and Bengio [**?**], introduces a non-saturating threshold operator:

$$\mathrm{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0, \\ 0 & \text{if } z \leq 0. \end{cases} \tag{1.52}$$

Its derivative is piecewise constant:

$$\frac{d\mathrm{ReLU}(z)}{dz} = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z < 0, \end{cases} \tag{1.53}$$

with the sub-gradient convention setting the derivative to 0 (or 0.5) at the non-differentiable point $z = 0$.

The advantages of ReLU over sigmoidal activations are structural:

(i) **Non-saturating gradient flow**: For positive pre-activations ($z > 0$), the derivative is identically equal to 1.0. The gradient passes through the unit without attenuation, mitigating the vanishing gradient problem across deep feed-forward and convolutional networks.

(ii) **Computational efficiency**: Evaluating $\max(0, z)$ requires only a comparison and conditional assignment, avoiding expensive floating-point exponentials.

(iii) **Induced sparsity**: When $z \leq 0$, the unit outputs exactly zero. In large networks, a significant proportion of units are inactive for any given input, yielding sparse representations that improve interpretability and computational economy.

The dying ReLU pathology. Despite these advantages, ReLU networks suffer from the "dying ReLU" problem [**?**]. If a large gradient update shifts a neuron's incoming weights and bias such that its pre-activation $z = \mathbf{w}^\top \mathbf{x} + b$ is negative for all training examples, the unit outputs zero and its derivative evaluates to zero across the entire dataset:

$$\forall \mathbf{x} \in \mathcal{D}_{\text{train}}, \quad z(\mathbf{x}) < 0 \implies \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}}{\partial z}\mathbf{x} = \mathbf{0}. \tag{1.54}$$

Once a unit enters this regime, gradient descent cannot provide non-zero updates to alter its incoming weights, permanently deactivating the neuron.

Leaky rectified linear unit and variants

To eliminate the dead state of standard ReLUs, the Leaky ReLU assigns a small, fixed positive slope $\alpha > 0$ to the negative half-plane:

$$\mathrm{LReLU}(z) = \max(\alpha z, z) = \begin{cases} z & \text{if } z > 0, \\ \alpha z & \text{if } z \leq 0, \end{cases} \tag{1.55}$$

typically parameterized with $\alpha = 0.01$. Its derivative evaluates to:

$$\frac{d\mathrm{LReLU}(z)}{dz} = \begin{cases} 1 & \text{if } z > 0, \\ \alpha & \text{if } z \leq 0. \end{cases} \tag{1.56}$$

Because the gradient remains non-zero ($\alpha > 0$) for negative inputs, neurons receive corrective gradients even when inactive, preventing permanent deactivation. In Parametric ReLU (PReLU), the slope α is treated as a learnable parameter optimized jointly via backpropagation.

Gaussian error linear unit

The Gaussian Error Linear Unit (GELU), formulated by Hendrycks and Gimpel in 2016 [**?**], serves as the standard activation function across modern foundation models, including BERT, GPT-2, and GPT-3. GELU integrates stochastic regularization directly into a deterministic activation function by weighting inputs by their probability under a standard Gaussian cumulative distribution:

$$\text{GELU}(z) = z\Phi(z) = zP(X \leq z), \quad \text{where} \quad X \sim \mathcal{N}(0,1). \tag{1.57}$$

The cumulative normal distribution $\Phi(z)$ is defined by the error function erf:

$$\Phi(z) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{z} e^{-\frac{t^2}{2}} dt = \frac{1}{2}\left[1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right]. \tag{1.58}$$

For efficient hardware execution, GELU is frequently evaluated using a fast cubic approximation:

$$\text{GELU}(z) \approx 0.5z\left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(z + 0.044715z^3\right)\right)\right). \tag{1.59}$$

Unlike ReLU, GELU is non-monotonic, smooth, and continuously differentiable everywhere. For negative inputs, GELU exhibits a small curvature dip with a local minimum at $z \approx -0.7517$ where $f(z) \approx -0.170$, before smoothly approaching zero as $z \to -\infty$. This smooth transition allows small negative activations to convey gradient information while suppressing large negative outliers.

SwiGLU and gated linear units

Modern Large Language Models (such as LLaMA, Mistral, and PaLM) employ Gated Linear Unit (GLU) variants within their feed-forward blocks, specifically SwiGLU [**?**]. A standard GLU computes the element-wise product of two linear transformations, one of which is modulated by a gating activation:

$$\text{GLU}(\mathbf{x}, \mathbf{W}, \mathbf{V}, \mathbf{b}, \mathbf{c}) = (\mathbf{x}\mathbf{W} + \mathbf{b}) \otimes \sigma\left(\mathbf{x}\mathbf{V} + \mathbf{c}\right), \tag{1.60}$$

where $\otimes$ denotes the element-wise Hadamard product. In SwiGLU, the logistic sigmoid gate is replaced by the Swish activation ($\text{Swish}_\beta(z) = z\sigma(\beta z)$ with $\beta = 1$):

$$\text{SwiGLU}(\mathbf{x}) = (\mathbf{x}\mathbf{W}) \otimes \text{Swish}\left(\mathbf{x}\mathbf{V}\right). \tag{1.61}$$

This bilinear multiplicative interaction enables hidden representations to dynamically modulate individual feature channels, yielding superior convergence stability and empirical language modeling performance, offset by a 1.5× parameter expansion within the intermediate MLP projection.

The softmax operator

When neural network outputs represent categorical probability distributions over C mutually exclusive classes, the scalar unconstrained pre-activation logits $\mathbf{z} = [z_1, z_2, \ldots, z_C]^\top \in \mathbb{R}^C$ must be mapped onto the probability simplex:

$$\Delta^{C-1} = \left\{ \mathbf{p} \in \mathbb{R}^C \;\middle|\; p_i \geq 0,\ \sum_{i=1}^{C} p_i = 1 \right\}. \tag{1.62}$$

This transformation is performed by the Softmax operator:

$$\hat{y}_i = \text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}. \tag{1.63}$$

Because the exponential function is strictly positive ($e^{z_i} > 0$), all output probabilities are strictly positive, and normalizing by the partition function $\sum_j e^{z_j}$ guarantees unit sum.

To compute the Jacobian matrix of the Softmax operator, we evaluate the partial derivative of output $\hat{y}_i$ with respect to logit z_k:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \frac{\partial}{\partial z_k}\left(\frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}\right). \tag{1.64}$$

Applying the quotient rule, we distinguish two cases. When $i = k$:

$$\frac{\partial \hat{y}_i}{\partial z_i} = \frac{e^{z_i}\sum_j e^{z_j} - e^{z_i}e^{z_i}}{\left(\sum_j e^{z_j}\right)^2} = \hat{y}_i - \hat{y}_i^2 = \hat{y}_i(1 - \hat{y}_i). \tag{1.65}$$

When $i \neq k$:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \frac{0 \cdot \sum_j e^{z_j} - e^{z_i}e^{z_k}}{\left(\sum_j e^{z_j}\right)^2} = -\hat{y}_i\hat{y}_k. \tag{1.66}$$

Combining both cases using the Kronecker delta δ_{ik} yields the compact Jacobian formulation:

$$\frac{\partial \hat{y}_i}{\partial z_k} = \hat{y}_i\left(\delta_{ik} - \hat{y}_k\right). \tag{1.67}$$

Numerical stability via shift invariance. Direct floating-point evaluation of Equation (1.63) causes numerical overflow if $z_i \gg 0$ (e.g., $e^{1000} \to \infty$ yielding `NaN`) or underflow if all $z_i \ll 0$. The Softmax operator satisfies the shift-invariance property: for any arbitrary scalar constant $c \in \mathbb{R}$:

$$\text{softmax}(\mathbf{z} + c)_i = \frac{e^{z_i + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}} = \text{softmax}(\mathbf{z})_i. \tag{1.68}$$

In practical implementations, numerical stability is guaranteed by choosing $c = -\max_k z_k$. The largest exponentiated term evaluates to $e^0 = 1.0$, which prevents overflow while ensuring the denominator contains at least one non-vanishing term.

1.4 Loss Functions and Optimization

To train a parameterized neural network, one must define a scalar criterion that quantifies the discrepancy between model predictions and empirical ground truth. This objective, termed the loss function, provides the supervisory signal that guides parameter optimization.

1.4.1 The operational role of loss functions

Formally, let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denote a training dataset sampled from an underlying data-generating distribution $P_{\text{data}}(\mathbf{x}, y)$. A neural network parameterized by $\theta \in \mathbb{R}^P$ evaluates a forward prediction mapping $\hat{y}_i = f(\mathbf{x}_i; \theta)$. The loss function $\mathcal{L}(\hat{y}_i, y_i) \in \mathbb{R}_{\geq 0}$ assigns a non-negative scalar penalty to prediction errors.

Under the framework of Empirical Risk Minimization (ERM), the training objective is to discover a parameter configuration θ^* that minimizes the empirical risk across the observed sample:

$$\theta^* = \arg\min_\theta \mathcal{R}_{\text{emp}}(\theta) = \arg\min_\theta \left[\frac{1}{N} \sum_{i=1}^N \mathcal{L}\left(f(\mathbf{x}_i; \theta), y_i\right) \right]. \tag{1.69}$$

The loss function serves as the sole mathematical feedback channel through which parameter errors are communicated to optimization algorithms. Without a well-defined scalar loss, the system possesses no objective measure of performance and cannot formulate directional gradient vectors for parameter improvement.

1.4.2 Task-conditioned output topologies

A fundamental principle of statistical machine learning is that a single universal loss function cannot exist. Different tasks operate across distinct mathematical domains:

(i) **Continuous regression**: Target variables span the unconstrained real line $y \in \mathbb{R}$ or vector spaces $\mathbb{R}^K$.

(ii) **Binary classification**: Targets represent dichotomous categorical indicators $y \in \{0, 1\}$.

(iii) **Multi-class classification**: Targets represent discrete class memberships $y \in \{1, \ldots, C\}$, geometrically residing on the probability simplex Δ^{C-1}.

To design mathematically principled loss functions across these diverse regimes, Simon Prince [**?**] formulates the Maximum Likelihood Estimation (MLE) recipe:

Step 1: Define an appropriate conditional probability distribution $P(y \mid \mathbf{x}; \phi)$ over the target space, parameterized by vector ϕ.

Step 2: Configure the neural network to output the parameters $\phi = f(\mathbf{x}; \theta)$ of that chosen distribution (e.g., predicting mean μ for a Gaussian, or probability p for a Bernoulli).

Step 3: Train the network to maximize the log-likelihood of the observed dataset, or equivalently, minimize the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\sum_{i=1}^N \log P\left(y_i \,\middle|\, \mathbf{x}_i; \theta\right). \tag{1.70}$$

This recipe ensures that the loss function is mathematically consistent with the output geometry and noise distribution of the underlying task.

1.4.3 Task-specific loss taxonomy and statistical derivations

Applying the maximum likelihood recipe across canonical distributions produces the standard loss taxonomy employed in modern neural networks.

Regression via Gaussian noise models

In continuous regression tasks, we assume the true target $y \in \mathbb{R}$ differs from the model's deterministic prediction $\hat{y} = f(\mathbf{x}; \theta)$ by additive zero-mean Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$:

$$y = f(\mathbf{x}; \theta) + \epsilon \implies P(y \mid \mathbf{x}; \theta, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(\mathbf{x}; \theta))^2}{2\sigma^2}\right). \tag{1.71}$$

Evaluating the negative log-likelihood over N independently and identically distributed observations yields:

$$\begin{aligned} \mathcal{L}_{\text{NLL}}(\theta) &= -\sum_{i=1}^{N} \log\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f(\mathbf{x}_i; \theta))^2}{2\sigma^2}\right)\right] \\ &= \frac{N}{2} \log\left(2\pi\sigma^2\right) + \frac{1}{2\sigma^2} \sum_{i=1}^{N} \left(y_i - f(\mathbf{x}_i; \theta)\right)^2. \end{aligned} \tag{1.72}$$

Assuming homoscedastic noise (σ^2 constant), discarding constant terms, and scaling by $\frac{1}{N}$ recovers the standard Mean Squared Error (MSE) objective:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \hat{y}_i\right)^2. \tag{1.73}$$

If the noise distribution is modeled instead as Laplacian noise $\epsilon \sim \text{Laplace}(0, b)$ with density $p(\epsilon) = \frac{1}{2b} \exp(-|\epsilon|/b)$, minimizing the negative log-likelihood recovers the Mean Absolute Error (MAE) or L_1 loss:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^{N} \left|y_i - \hat{y}_i\right|, \tag{1.74}$$

which exhibits enhanced robustness to dataset outliers because penalties scale linearly rather than quadratically.

Binary classification via Bernoulli distributions

For binary classification ($y \in \{0, 1\}$), the target is modeled as a Bernoulli random variable conditioned on the input:

$$P(y \mid \mathbf{x}) = \hat{y}^y \left(1 - \hat{y}\right)^{1-y}, \quad \text{where} \quad \hat{y} = \sigma(z) \in (0, 1). \tag{1.75}$$

Evaluating the negative log-likelihood of this distribution yields the Binary Cross-Entropy (BCE) loss:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\right]. \tag{1.76}$$

The expression functions as a ground-truth selector:

(i) When ground-truth $y_i = 1$, the second term $(1 - y_i)$ vanishes identically, reducing the penalty to $-\log \hat{y}_i$. The loss approaches 0 as $\hat{y}_i \to 1$, but penalizes incorrect predictions with an asymptote approaching $+\infty$ as $\hat{y}_i \to 0$.

(ii) When ground-truth $y_i = 0$, the first term vanishes, reducing the penalty to $-\log(1 - \hat{y}_i)$, which penalizes predictions approaching $\hat{y}_i \to 1$ with infinite cost.

Multi-class classification via categorical distributions

In multi-class classification over C mutually exclusive categories ($y \in \{1, \ldots, C\}$), targets are represented as one-hot vectors $\mathbf{y} \in \{0,1\}^C$ where $\sum_{k=1}^{C} y_k = 1$. The network predicts a categorical distribution $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \in \Delta^{C-1}$. The conditional likelihood is given by the multinomial density:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{k=1}^{C} \hat{y}_k^{y_k}. \tag{1.77}$$

Minimizing the negative log-likelihood yields the Categorical Cross-Entropy (CCE) loss:

$$\mathcal{L}_{\text{CCE}} = -\sum_{k=1}^{C} y_k \log \hat{y}_k = -\log \hat{y}_c, \tag{1.78}$$

where $c \in \{1, \ldots, C\}$ denotes the index of the true positive class ($y_c = 1$).

Analytical proof of MSE classification failure

A common pedagogical error involves applying Mean Squared Error to classification tasks equipped with sigmoidal outputs. To demonstrate why this combination fails, we evaluate the gradient of the loss with respect to the pre-activation logit z for a binary classifier ($\hat{y} = \sigma(z)$) under both MSE and BCE.

Case 1: MSE logit gradient. The MSE loss for a single observation is $\mathcal{L}_{\text{MSE}} = (y - \hat{y})^2$. Applying the chain rule:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{d\hat{y}}{dz} = -2\left(y - \hat{y}\right)\sigma'(z) = -2\left(y - \hat{y}\right)\hat{y}\left(1 - \hat{y}\right). \tag{1.79}$$

Now, evaluate the gradient when the model is confident and wrong: suppose the true label is $y = 1$, but the network currently produces a strongly negative logit $z = -5$, such that $\hat{y} = \sigma(-5) \approx 0.0067$. The residual error $(y - \hat{y}) = 1 - 0.0067 = 0.9933$ is maximal. However, the derivative evaluates to:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -2(0.9933) \cdot (0.0067)(0.9933) \approx -0.0132 \approx 0. \tag{1.80}$$

Because the sigmoid derivative saturates ($\sigma'(z) \to 0$ as $|z| \gg 0$), the gradient vanishes despite maximal prediction error. The parameter updates stall, preventing gradient descent from correcting catastrophic misclassifications.

Case 2: BCE logit gradient. Evaluating the logit gradient under Binary Cross-Entropy:

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial \hat{y}} \frac{d\hat{y}}{dz} \\
&= \left[-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right]\hat{y}\left(1-\hat{y}\right) \\
&= \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})}\hat{y}\left(1-\hat{y}\right) \\
&= -y + y\hat{y} + \hat{y} - y\hat{y} \\
&= \hat{y} - y.
\end{aligned} \tag{1.81}$$

Under cross-entropy, the saturating term $\hat{y}(1-\hat{y})$ in the denominator of $\frac{\partial \mathcal{L}}{\partial \hat{y}}$ cancels the identical term in the sigmoid derivative $\frac{d\hat{y}}{dz}$. The resulting gradient evaluates to the linear residual error

$\hat{y} - y$. If $y = 1$ and $\hat{y} \approx 0.0067$, the gradient evaluates to $\hat{y} - y = 0.0067 - 1 = -0.9933$, providing a strong, unattenuated corrective signal.

Furthermore, analyzing the second derivative demonstrates that $\frac{\partial^2 \mathcal{L}_{\text{MSE}}}{\partial z^2}$ flips signs across logit space, creating non-convex regions and saddle plateaus [**?**]. Conversely, for BCE:

$$\frac{\partial^2 \mathcal{L}_{\text{BCE}}}{\partial z^2} = \frac{d}{dz}\left(\hat{y} - y\right) = \sigma'(z) = \hat{y}\left(1 - \hat{y}\right) > 0, \quad \forall z \in \mathbb{R}. \tag{1.82}$$

The strictly positive second derivative proves that Binary Cross-Entropy is strictly convex with respect to the output logits, guaranteeing that no non-convex plateaus impede convergence.

1.4.4 The analytical intractability of direct root finding

In classical linear regression with quadratic loss, the parameter optimization problem admits an exact, closed-form analytical solution. Consider an input data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ and target vector $\mathbf{y} \in \mathbb{R}^N$. The quadratic loss function is:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 . \tag{1.83}$$

To minimize this loss, one computes the gradient with respect to $\mathbf{w}$ and sets it equal to the zero vector:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}. \tag{1.84}$$

This condition reduces to the system of linear equations:

$$\mathbf{X}^\top \mathbf{X}\mathbf{w} = \mathbf{X}^\top \mathbf{y}, \tag{1.85}$$

which, assuming $\mathbf{X}^\top \mathbf{X}$ is non-singular, yields the Ordinary Least Squares (OLS) normal equations:

$$\mathbf{w}^* = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}. \tag{1.86}$$

In sharp contrast, setting $\nabla_\theta \mathcal{L}(\theta) = \mathbf{0}$ fails completely for deep neural networks. In multi-layer architectures, the model output is constructed through nested compositions of affine transformations and non-linear activation functions:

$$\hat{y} = f(\mathbf{x}; \theta) = \mathbf{W}_L^\top \sigma\left(\mathbf{W}_{L-1}^\top \dots \sigma\left(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_{L-1}\right) + \mathbf{b}_L. \tag{1.87}$$

When differentiated, the parameters $\theta = \{\mathbf{W}_l, \mathbf{b}_l\}_{l=1}^L$ appear inside complex transcendental functions (exponentials, piecewise steps, products). Setting the resulting gradient equations to zero yields a coupled system of non-linear transcendental algebraic equations with millions of unknowns:

$$\nabla_\theta \mathcal{L}(\theta) = \sum_{i=1}^{N} \frac{\partial \mathcal{L}_i}{\partial f_i} \nabla_\theta f(\mathbf{x}_i; \theta) = \mathbf{0}. \tag{1.88}$$

This system possesses no closed-form analytical root. Moreover, the loss surface $\mathcal{L}(\theta)$ is highly non-convex, characterized by complex landscapes with multiple local minima, degenerate saddle points, and ill-conditioned ravines. Consequently, deep neural networks cannot be solved analytically; their optimization requires iterative, numerical gradient-based methods.

1.4.5 Gradient descent and modern optimization heuristics

Iterative numerical optimization computes successive parameter updates by evaluating local first-order gradient vectors.

Vanilla gradient descent

The foundational iterative algorithm is Gradient Descent (GD). At each iteration t, the algorithm updates parameter vector θ_t by taking a step proportional to the negative gradient of the empirical risk:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t), \tag{1.89}$$

where $\alpha > 0$ denotes the learning rate hyperparameter.

While intuitive, vanilla gradient descent encounters three severe operational pathologies:

(i) **Sensitivity to learning rate**: If α is chosen too small, parameter updates crawl inefficiently, requiring prohibitive computational time. Conversely, if α exceeds the reciprocal of the maximum eigenvalue of the Hessian ($\alpha > \frac{2}{\lambda_{\max}}$), the updates overshoot valleys and diverge.

(ii) **Oscillations in ill-conditioned ravines**: When the loss surface exhibits high condition numbers (where curvature along one axis is drastically steeper than along another, $\lambda_{\max} \gg \lambda_{\min}$), gradient descent oscillates violently across the steep walls while making negligible progress along the flat base.

(iii) **Stagnation at saddle points**: In high-dimensional optimization landscapes, local minima are rare; the dominant topological features are saddle points where the gradient vanishes ($\nabla_\theta \mathcal{L} \approx \mathbf{0}$) but some directions exhibit negative curvature. Vanilla GD decelerates severely in the vicinity of saddle plateaus.

Momentum

To dampen high-frequency oscillations across steep ravine walls and accelerate descent along flat valleys, Polyak (1964) introduced the momentum method [**?**]. Momentum simulates physical inertia by accumulating an exponentially decaying moving average of past gradients into a velocity vector $\mathbf{v}_t$:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1-\beta)\nabla_\theta \mathcal{L}(\theta_t), \tag{1.90}$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_t, \tag{1.91}$$

where $\beta \in [0, 1)$ represents the momentum decay coefficient (typically $\beta = 0.9$). Along directions where the gradient repeatedly reverses sign, the oscillating terms cancel out in the running average, dampening instability. Along directions where the gradient consistently points in the same direction, the velocity accumulates to an effective magnitude $\frac{1}{1-\beta}$ times larger than the instantaneous gradient, accelerating convergence through plateaus.

Adam: Adaptive Moment Estimation

In deep architectures and large language models, different parameters operate at widely disparate scales and receive gradients of vastly differing frequencies (e.g., embeddings of rare tokens receive sparse, large updates). Adaptive learning rate algorithms assign individual learning rates to each parameter.

The universal optimization standard across foundation models is Adam (Adaptive Moment Estimation), introduced by Kingma and Ba in 2014 [**?**]. Adam tracks both the exponentially decaying average of past gradients (first raw moment) and the exponentially decaying average of squared gradients (second uncentered moment):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \tag{1.92}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \tag{1.93}$$

where $\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t)$, and $\mathbf{g}_t^2$ denotes the element-wise Hadamard square $\mathbf{g}_t \odot \mathbf{g}_t$. The decay hyperparameters are standardly initialized to $\beta_1 = 0.9$ and $\beta_2 = 0.999$.

Because $\mathbf{m}_0$ and $\mathbf{v}_0$ are initialized to the zero vector $\mathbf{0}$, the moment vectors are biased toward zero, particularly during initial training steps when β_1 and β_2 are close to 1. To correct for this initialization artifact, Adam applies analytical bias corrections:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad \text{and} \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \tag{1.94}$$

The parameter update is then evaluated coordinate-wise:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \hat{\mathbf{m}}_t, \tag{1.95}$$

where $\epsilon \approx 10^{-8}$ prevents division by zero. Adam dynamically rescales updates: parameters with large historical variance receive smaller, dampened steps, while parameters with sparse, small gradients receive amplified steps, ensuring robust optimization dynamics across diverse neural layers.

1.5 Computational Graphs and Backpropagation

To execute gradient-based optimization algorithms such as Adam, the learning system must compute the partial derivatives of the scalar loss function with respect to every parameter in the network ($\nabla_\theta \mathcal{L}$). This section establishes the graph-theoretic formalism of computational graphs, derives the reverse-mode automatic differentiation algorithm, and synthesizes these mechanisms into a complete multi-layer feed-forward network.

1.5.1 Methods of gradient evaluation

As formalized by Christopher Bishop and Hugh Bishop (2024, Ch. 8) [**?**], there exist four distinct methodological approaches to computing derivatives in numerical systems:

(i) **Analytical differentiation**: The manual derivation of derivative expressions by hand using pencil and paper calculus. While mathematically exact, this approach is rigid, labor-intensive, and prone to human algebraic errors. Furthermore, modifying network architecture requires completely re-deriving the analytical formulas from scratch.

(ii) **Numerical differentiation (finite differences)**: The numerical approximation of partial derivatives via perturbation formulas:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta + \epsilon \mathbf{e}_i) - \mathcal{L}(\theta)}{\epsilon}, \tag{1.96}$$

where $\mathbf{e}_i$ denotes the i-th standard basis vector, and $\epsilon \approx 10^{-5}$. While universally applicable to black-box functions, numerical differentiation is computationally prohibitive: evaluating the gradient across a model with D parameters requires $D + 1$ complete forward passes, yielding an asymptotic computational complexity of $O(D)$. For modern foundation models where $D \geq 10^9$, computing a single gradient step would require over one billion forward passes. Furthermore, finite differences suffer from numerical roundoff and truncation errors.

(iii) **Symbolic differentiation**: Computer algebra systems (such as Mathematica or SymPy) construct symbolic mathematical expressions for derivatives using calculus rules. While exact, symbolic differentiation treats mathematical expressions as abstract trees and suffers from exponential *expression swell*: redundant subexpressions are duplicated repeatedly across the expression tree, resulting in massive computational overhead. Moreover, symbolic engines cannot easily differentiate algorithms containing procedural control flow (loops, branches, and conditional recursion).

(iv) **Automatic differentiation**: The algorithmic evaluation of exact numerical derivatives by decomposing complex functions into sequences of elementary mathematical primitives whose derivatives are known. Rather than manipulating symbolic expressions, automatic differentiation evaluates numerical derivatives directly during program execution, providing exact gradients at machine precision with computational cost proportional to the forward function evaluation.

1.5.2 Computational graphs and reverse-mode automatic differentiation

Modern deep learning frameworks implement automatic differentiation by representing neural network execution as a Directed Acyclic Graph (DAG) $G = (\mathcal{V}, \mathcal{E})$ [**? ?**].

(i) The set of vertices $\mathcal{V}$ represents computational operations (such as $+$, $\times$, exp, tanh) or intermediate tensor variables.

(ii) The set of directed edges $\mathcal{E}$ represents the directional flow of tensor values, where a directed edge $(u, v) \in \mathcal{E}$ indicates that node v directly depends on the value computed by node u.

Forward evaluation corresponds to traversing the graph according to a topological sort: any node v can be evaluated once all its incoming parent nodes parents(v) have completed execution.

Reverse-mode automatic differentiation (Backpropagation). To compute gradients, reverse-mode automatic differentiation traverses the computational graph in reverse topological order, starting from the final scalar loss $\mathcal{L}$ and propagating gradient signals backward toward the input and parameter leaves.

Following the adjoint notation introduced by Linnainmaa (1970) and Rumelhart et al. (1986) [**?**], we define the adjoint $\bar{u}$ of any intermediate node u as the partial derivative of the final scalar loss with respect to u:

$$\bar{u} \equiv \frac{\partial \mathcal{L}}{\partial u}. \tag{1.97}$$

The multivariable chain rule of calculus states that if a variable u influences the loss $\mathcal{L}$ through multiple downstream dependent nodes $v \in \text{children}(u)$, the total sensitivity $\bar{u}$ is obtained by summing the contributions transmitted through all outgoing edges:

$$\bar{u} = \sum_{v \in \text{children}(u)} \bar{v} \frac{\partial v}{\partial u} = \sum_{v \in \text{children}(u)} \frac{\partial \mathcal{L}}{\partial v} \frac{\partial v}{\partial u}. \tag{1.98}$$

The computational efficiency of reverse-mode automatic differentiation is profound. Consider a function mapping D parameters to a single scalar loss: $f : \mathbb{R}^D \to \mathbb{R}$.

(i) **Forward-mode automatic differentiation** computes directional derivatives (tangents $\dot{v} = \frac{\partial v}{\partial \theta_i}$) forward from the inputs. Computing the gradient with respect to all D parameters requires D distinct forward sweeps, scaling as $O(D)$.

(ii) **Reverse-mode automatic differentiation** initiates a single adjoint evaluation at the scalar loss ($\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1.0$) and sweeps backward across the graph in reverse topological order. All D partial derivatives are evaluated simultaneously in a single backward pass.

The time complexity of the reverse sweep is bounded by $O(|\mathcal{V}| + |\mathcal{E}|)$, which is approximately equal to twice the computational cost of the forward pass ($T_{\text{backward}} \approx 2 \times T_{\text{forward}}$), completely independent of the parameter dimension D. This fundamental efficiency renders backpropagation the engine of deep learning.

1.5.3 Worked arithmetic calculation of backpropagation

To demystify the abstract multivariable calculus of reverse-mode autodiff, we reconstruct the exact step-by-step arithmetic trace demonstrated in the course lecture slides.

Consider a single training observation $(x, y) \in \mathbb{R} \times \mathbb{R}$. The model computes a prediction $\hat{y}$ through the product of two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$:

$$\hat{y} = \theta_1 \theta_2 x. \tag{1.99}$$

The optimization objective is the scalar squared error loss:

$$\mathcal{L} = (\hat{y} - y)^2 = (\theta_1 \theta_2 x - y)^2. \tag{1.100}$$

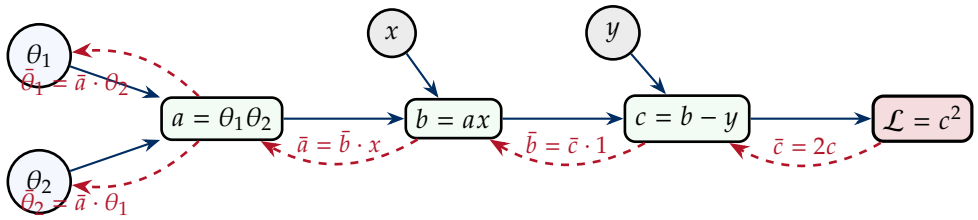


Figure 1.4: Computational graph representation for the lecture worked example $\mathcal{L} = (\theta_1\theta_2 x - y)^2$. Solid blue arrows indicate forward topological execution; dashed red arrows trace the reverse-mode adjoint propagation.

Forward evaluation trace. The computational graph decomposes this loss into a sequence of four elementary intermediate operations:

$$\text{Node } a = \theta_1\theta_2, \tag{1.101}$$
$$\text{Node } b = a \cdot x = \theta_1\theta_2 x, \tag{1.102}$$
$$\text{Node } c = b - y = \theta_1\theta_2 x - y, \tag{1.103}$$
$$\text{Node } \mathcal{L} = c^2 = \left(\theta_1\theta_2 x - y\right)^2. \tag{1.104}$$

Backward adjoint propagation trace. In the reverse pass, we evaluate the adjoints $\bar{u} = \frac{\partial \mathcal{L}}{\partial u}$ in reverse topological order:

Step 1: **Initialize root adjoint**: At the root node, the derivative of the loss with respect to itself is identically unity:

$$\bar{\mathcal{L}} = \frac{\partial \mathcal{L}}{\partial \mathcal{L}} = 1. \tag{1.105}$$

Step 2: **Adjoint of node** c: Differentiating $\mathcal{L} = c^2$ with respect to c:

$$\bar{c} = \frac{\partial \mathcal{L}}{\partial c} = \bar{\mathcal{L}}\frac{\partial \mathcal{L}}{\partial c} = 1 \cdot (2c) = 2\left(\theta_1\theta_2 x - y\right). \tag{1.106}$$

Step 3: **Adjoint of node** b: Differentiating $c = b - y$ with respect to b:

$$\bar{b} = \frac{\partial \mathcal{L}}{\partial b} = \bar{c}\frac{\partial c}{\partial b} = \bar{c} \cdot 1 = 2\left(\theta_1\theta_2 x - y\right). \tag{1.107}$$

Step 4: **Adjoint of node** a: Differentiating $b = a \cdot x$ with respect to a:

$$\bar{a} = \frac{\partial \mathcal{L}}{\partial a} = \bar{b}\frac{\partial b}{\partial a} = \bar{b} \cdot x = \left[2\left(\theta_1\theta_2 x - y\right)\right] x = 2x\left(\theta_1\theta_2 x - y\right). \tag{1.108}$$

Step 5: **Adjoint of parameter** θ_1: Differentiating $a = \theta_1\theta_2$ with respect to θ_1:

$$\bar{\theta}_1 = \frac{\partial \mathcal{L}}{\partial \theta_1} = \bar{a}\frac{\partial a}{\partial \theta_1} = \bar{a} \cdot \theta_2 = \left[2x\left(\theta_1\theta_2 x - y\right)\right]\theta_2 = 2\theta_2 x\left(\theta_1\theta_2 x - y\right). \tag{1.109}$$

Step 6: **Adjoint of parameter** θ_2: Differentiating $a = \theta_1\theta_2$ with respect to θ_2:

$$\bar{\theta}_2 = \frac{\partial \mathcal{L}}{\partial \theta_2} = \bar{a}\frac{\partial a}{\partial \theta_2} = \bar{a} \cdot \theta_1 = \left[2x\left(\theta_1\theta_2 x - y\right)\right]\theta_1 = 2\theta_1 x\left(\theta_1\theta_2 x - y\right). \tag{1.110}$$

With the exact partial derivatives computed, parameter updates under gradient descent evaluate to:

$$\theta_1 \leftarrow \theta_1 - \alpha\bar{\theta}_1 = \theta_1 - 2\alpha\theta_2 x\left(\theta_1\theta_2 x - y\right), \tag{1.111}$$

$$\theta_2 \leftarrow \theta_2 - \alpha\bar{\theta}_2 = \theta_2 - 2\alpha\theta_1 x\left(\theta_1\theta_2 x - y\right). \tag{1.112}$$

This minimal worked example illustrates the operational universality of reverse-mode automatic differentiation: regardless of model depth or topological branching, the global gradient is resolved into localized, modular partial derivative multiplications evaluated backward along computational edges.

1.5.4 The complete multi-layer feed-forward network

We now synthesize all established primitives—linear projections, non-linear activation functions, task-specific loss criteria, and reverse-mode automatic differentiation—into a unified, fully vectorized multi-layer Feed-Forward Network (FFN).

Vectorized layer equations

Consider an L-layer network. Let $\mathbf{x} \in \mathbb{R}^{d_0}$ denote the input vector, and define $\mathbf{a}^{(0)} \equiv \mathbf{x}$. For each layer $l \in \{1, 2, \ldots, L\}$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)\top}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f^{(l)}\left(\mathbf{z}^{(l)}\right), \tag{1.113}$$

where:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{d_{l-1}\times d_l} \quad \text{denotes the weight transformation matrix for layer } l, \tag{1.114}$$

$$\mathbf{b}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the scalar bias vector for layer } l, \tag{1.115}$$

$$\mathbf{z}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the pre-activation logit vector,} \tag{1.116}$$

$$\mathbf{a}^{(l)} \in \mathbb{R}^{d_l} \quad \text{denotes the post-activation latent feature representation,} \tag{1.117}$$

$$f^{(l)}(\cdot) \quad \text{denotes the element-wise non-linear activation function.} \tag{1.118}$$

The final output vector $\hat{\mathbf{y}} = \mathbf{a}^{(L)} \in \mathbb{R}^{d_L}$ produces predictions tailored to task topology (e.g., continuous predictions via linear activations for regression, or probability distributions via Softmax for classification).

Vectorized backpropagation equations

Let $\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}^*)$ denote the scalar loss evaluated on ground truth $\mathbf{y}^*$. In vectorized matrix calculus, we define the layer error vector $\boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}$ as the gradient of the loss with respect to the pre-activation vector $\mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} \equiv \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}. \tag{1.119}$$

Output layer error $\boldsymbol{\delta}^{(L)}$. Applying the chain rule to the output layer:

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}} = \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot f'^{(L)}\left(\mathbf{z}^{(L)}\right), \tag{1.120}$$

where $\odot$ denotes the element-wise Hadamard product. As proven in Equation (1.81), when pairing cross-entropy loss with Softmax or Sigmoid activations, this expression simplifies directly to the residual difference:

$$\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}^*. \tag{1.121}$$

Hidden layer error recursion $\boldsymbol{\delta}^{(l)}$. For any internal hidden layer $l \in \{1, \dots, L-1\}$, the error vector $\boldsymbol{\delta}^{(l)}$ is evaluated recursively backward from layer $l+1$:

$$\boldsymbol{\delta}^{(l)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \left(\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{z}^{(l)}} \right)^{\top} \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l+1)}} = \left(\mathbf{W}^{(l+1)} \boldsymbol{\delta}^{(l+1)} \right) \odot f'^{(l)} \left(\mathbf{z}^{(l)} \right). \tag{1.122}$$

Equation (1.122) demonstrates the structural symmetry of backpropagation: the error signal $\boldsymbol{\delta}^{(l+1)}$ is projected backward through the transpose of the forward weight matrix $\mathbf{W}^{(l+1)}$ and subsequently modulated by the local derivative of the activation function $f'^{(l)}(\mathbf{z}^{(l)})$.

Parameter gradient evaluation. Once the error vectors $\boldsymbol{\delta}^{(l)}$ are evaluated, the gradients of the loss with respect to the weight matrices and bias vectors are obtained via outer products:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \mathbf{a}^{(l-1)} \left(\boldsymbol{\delta}^{(l)} \right)^{\top} \in \mathbb{R}^{d_{l-1} \times d_l}, \tag{1.123}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}. \tag{1.124}$$

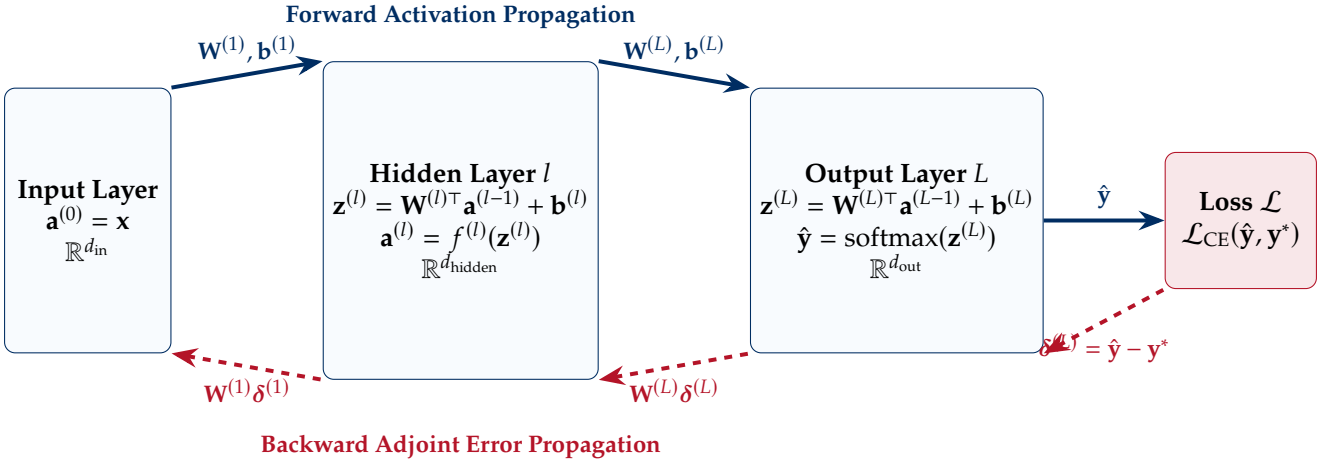


Figure 1.5: Vectorized dataflow schematic across a multi-layer Feed-Forward Network. Solid blue lines trace forward activations; dashed red lines trace the reverse-mode propagation of error vectors $\boldsymbol{\delta}^{(l)}$.

Structural limitations of feed-forward networks for sequence processing

While the multi-layer feed-forward network provides a universal framework for static pattern recognition, its fundamental architectural assumptions break down when deployed on sequential data, such as natural language text, source code, and time series [**? ?**]. Specifically, standard feed-forward networks exhibit three structural failure modes:

(i) **Fixed-dimensional input constraints**: An FFN requires an input vector of strictly fixed dimension d_{in}. However, natural language utterances and software engineering artifacts vary arbitrarily in length. Forcing variable-length sentences into a fixed vector through zero-padding or truncation either wastes computation on empty tokens or discards critical semantic context.

(ii) **Absence of temporal memory**: Feed-forward layers operate statelessly: each input vector $\mathbf{x}$ is processed in total isolation from preceding observations. The network possesses no persistent internal hidden state to store linguistic dependencies, resolve pronouns, or maintain scope across long programmatic contexts.

(iii) **Inability to share parameters across sequence positions**: In a standard linear layer $\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$, each input feature is assigned a unique, position-dependent weight. If an informative linguistic motif (such as the keyword `function` or the phrase *"not found"*) appears at token position $t = 1$, the network cannot recognize that identical motif at token position $t = 15$ without separately learning an independent set of weights for those coordinates.

Overcoming these limitations requires an architecture equipped with persistent internal states and position-invariant parameter sharing. This fundamental representational shift directly motivates the transition to Recurrent Neural Networks, explored in Chapter **??**.